Graph Generation

Contents

En esta sección, hablaremos de cómo generar un grafo usando la biblioteca. Específicamente, estudiaremos cómo crear grafos sintéticos, que no son grafos del mundo real pero que son útiles para probar y comparar algoritmos. Además, veremos algunas propiedades importantes de grafos que pueden usarse para analizar la estructura de un grafo. `networkx`

13.1. Grafos sintéticos

Un grafo sintético es un gráfico que se genera artificialmente, en lugar de ser extraído de datos del mundo real. Estos grafos son útiles para probar y comparar algoritmos, ya que nos permiten controlar

Saltar al contenido principal

sección, hablaremos de cómo generar grafos sintéticos usando la biblioteca. `networkx`

13.1.1. Gráficos aleatorios

Uno de los tipos más simples de grafos sintéticos es un grafo aleatorio. Un grafo aleatorio es un grafo en el que las aristas se generan de forma aleatoria, sujetas a ciertas restricciones. Existen varios modelos para generar grafos aleatorios, como el modelo de Erdős-Rényi y el modelo Barabási-Albert.

13.1.1.1. Modelo de Erdős-Rényi

El modelo de Erdős-Rényi es uno de los modelos más conocidos para generar grafos aleatorios. En este modelo, empezamos con un número fijo de nodos y añadimos aristas entre pares de nodos con cierta probabilidad. La biblioteca proporciona una función para generar grafos aleatorios usando el modelo de Erdős-Rényi. `networkx` `networkx.erdos_renyi_graph`

```
import networkx as nx
import matplotlib.pyplot as plt

# Generate a random graph with 10 nodes and ed
G = nx.erdos_renyi_graph(10, 0.3, seed=42)

# Draw the graph
```

Saltar al contenido principal

```
plt.title("Random Graph (Erdős-Rényi Model)")
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=300,
nx.draw_networkx_edges(G, pos, width=1.0, alpl
plt.axis("off")
plt.show()
```

Este código nos da el siguiente gráfico:

La salida del código anterior es la siguiente gráfica:

Saltar al contenido principal

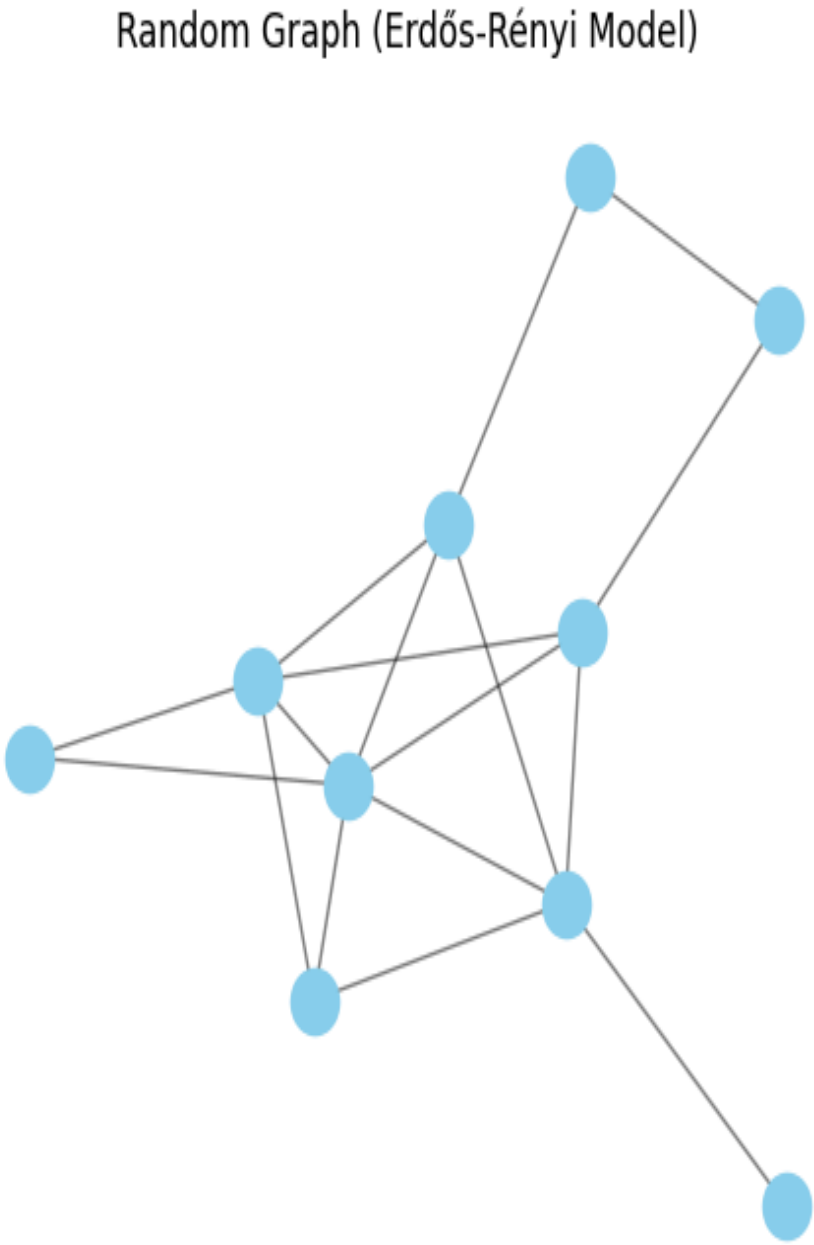

Fig. 13.1 Grafo aleatorio (modelo de Erdős-Rényi)

En el fragmento de código anterior, generamos un

aristas de 0,3 usando el modelo de Erdős-Rényi. Luego visualizamos el grafo usando la biblioteca. `matplotlib`

13.1.1.2. Modelo Barabási-Albert

El modelo Barabási-Albert es otro modelo popular para generar grafos aleatorios. En este modelo, comenzamos con un pequeño número de nodos y añadimos nuevos nodos con un mecanismo de unión preferencial, donde los nodos con grados superiores tienen más probabilidades de recibir nuevas aristas. La biblioteca proporciona una función para generar grafos aleatorios utilizando el modelo Barabási-Albert. `networkx` `networkx.barabasi_albert_graph`

```
# Generate a random graph with 10 nodes and pr
G = nx.barabasi_albert_graph(10, 2, seed=42)

# Draw the graph
plt.figure(figsize=(6, 6))
plt.title("Random Graph (Barabási-Albert Model
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=300,
nx.draw_networkx_edges(G, pos, width=1.0, alph
plt.axis("off")
plt.show()
```

Saltar al contenido principal

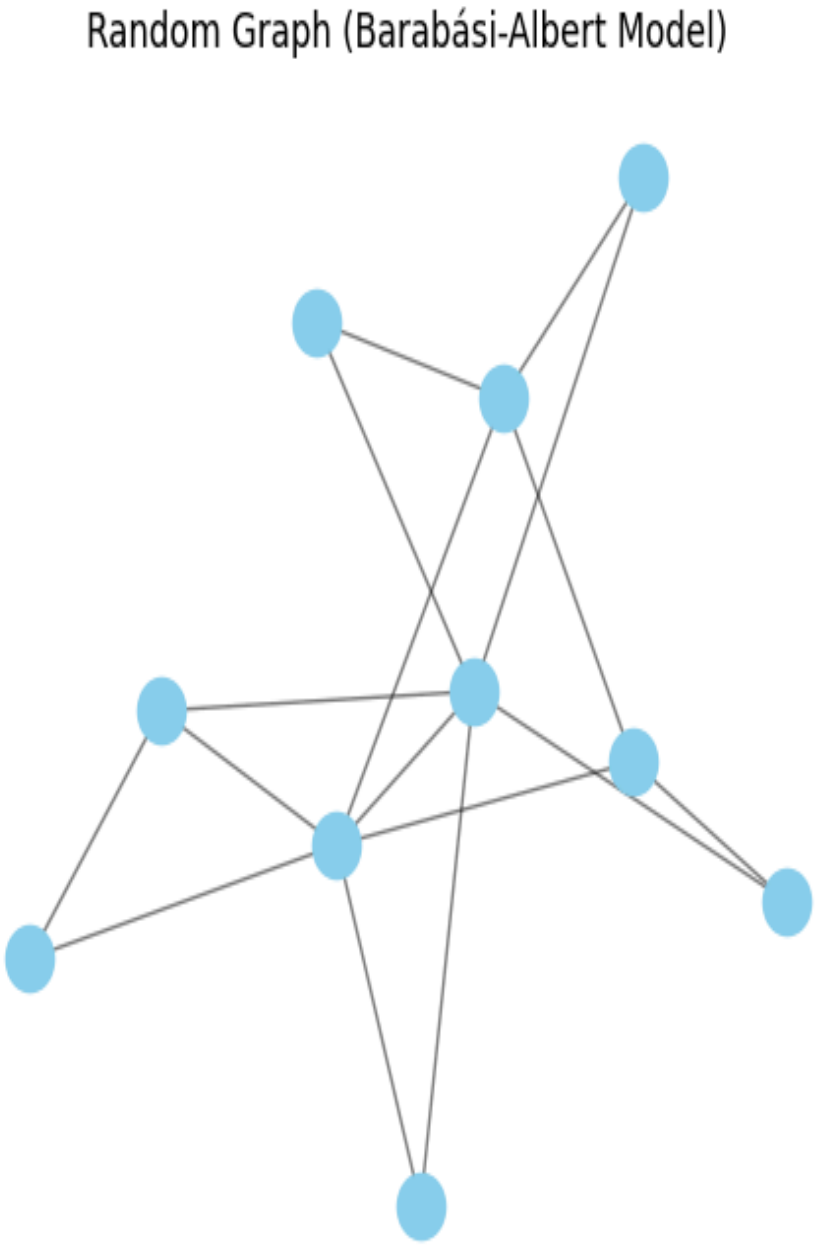

Fig. 13.2 Random Graph (Barabási-Albert Model)

In the code snippet above, we generate a random

parameter using the Barabási-Albert model. We then visualize the graph using the library. `m=2` `matplotlib`

13.1.2. Small-World Graphs

Another interesting type of synthetic graph is a small-world graph. Small-world graphs are characterized by having a small average shortest path length between nodes, similar to real-world social networks. The library provides a function to generate small-world graphs using the Watts-Strogatz model. `networkx` `networkx.watts_strogatz_graph`

```
# Generate a small-world graph with 10 nodes,
G = nx.watts_strogatz_graph(10, 4, 0.3)

# Draw the graph
plt.figure(figsize=(6, 6))
plt.title("Small-World Graph (Watts-Strogatz M
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=300,
nx.draw_networkx_edges(G, pos, width=1.0, alph
plt.axis("off")
plt.show()
```

This code gives us the following graph:

Saltar al contenido principal

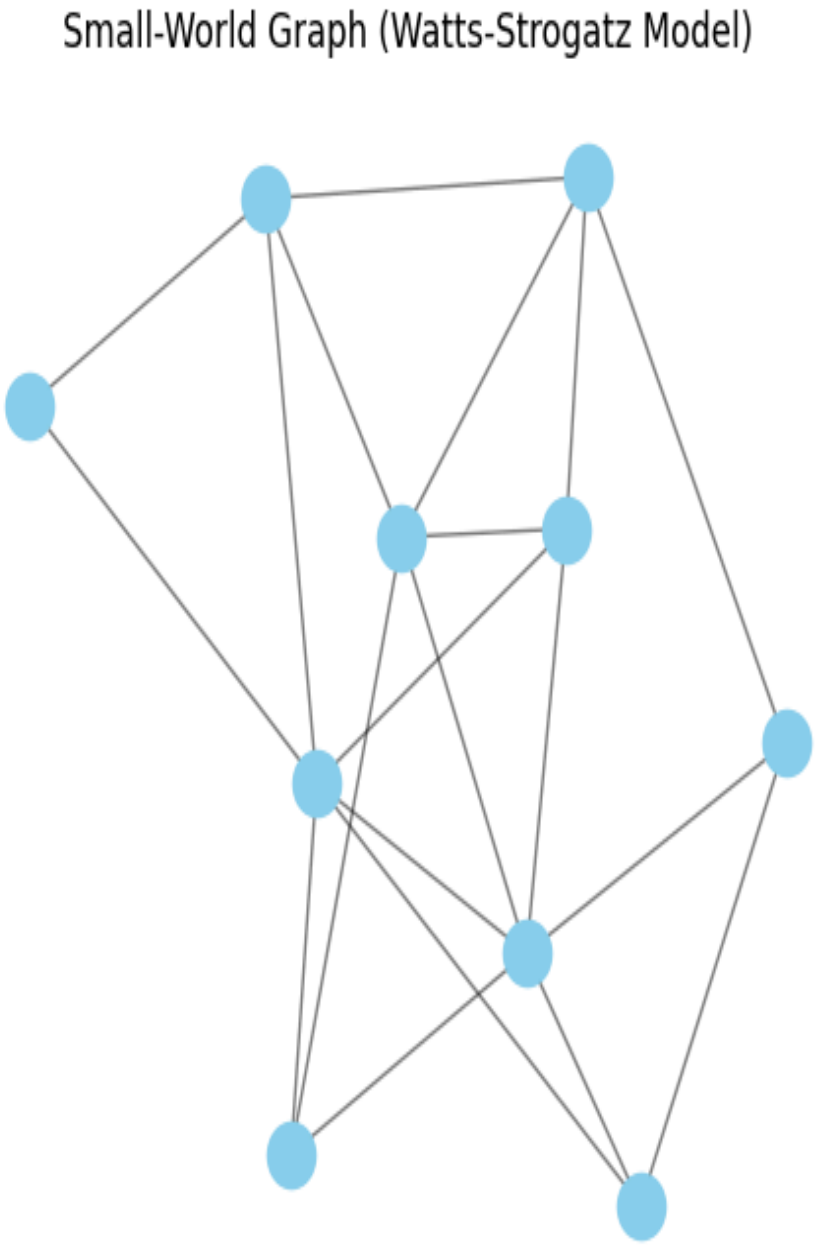

Fig. 13.3 Small-World Graph (Watts-Strogatz Model)

In the code snippet above, we generate a small-world

Saltar al contenido principal

rewiring probability of 0.3 using the Watts-Strogatz model. We then visualize the graph using the library. `matplotlib`

13.1.3. Tree Graphs

A tree is a connected acyclic graph, meaning that there are no cycles in the graph. Trees are a fundamental data structure in computer science and are used in various algorithms and data structures. The library provides a function to generate random tree graphs. `networkx` `networkx.random_tree`

```
# Generate a random tree graph with 10 nodes
G = nx.random_tree(10, seed=42)

# Draw the graph
plt.figure(figsize=(6, 6))
plt.title("Random Tree Graph")
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=300,
nx.draw_networkx_edges(G, pos, width=1.0, alpl
plt.axis("off")
plt.show()
```

This code gives us the following graph:

Saltar al contenido principal

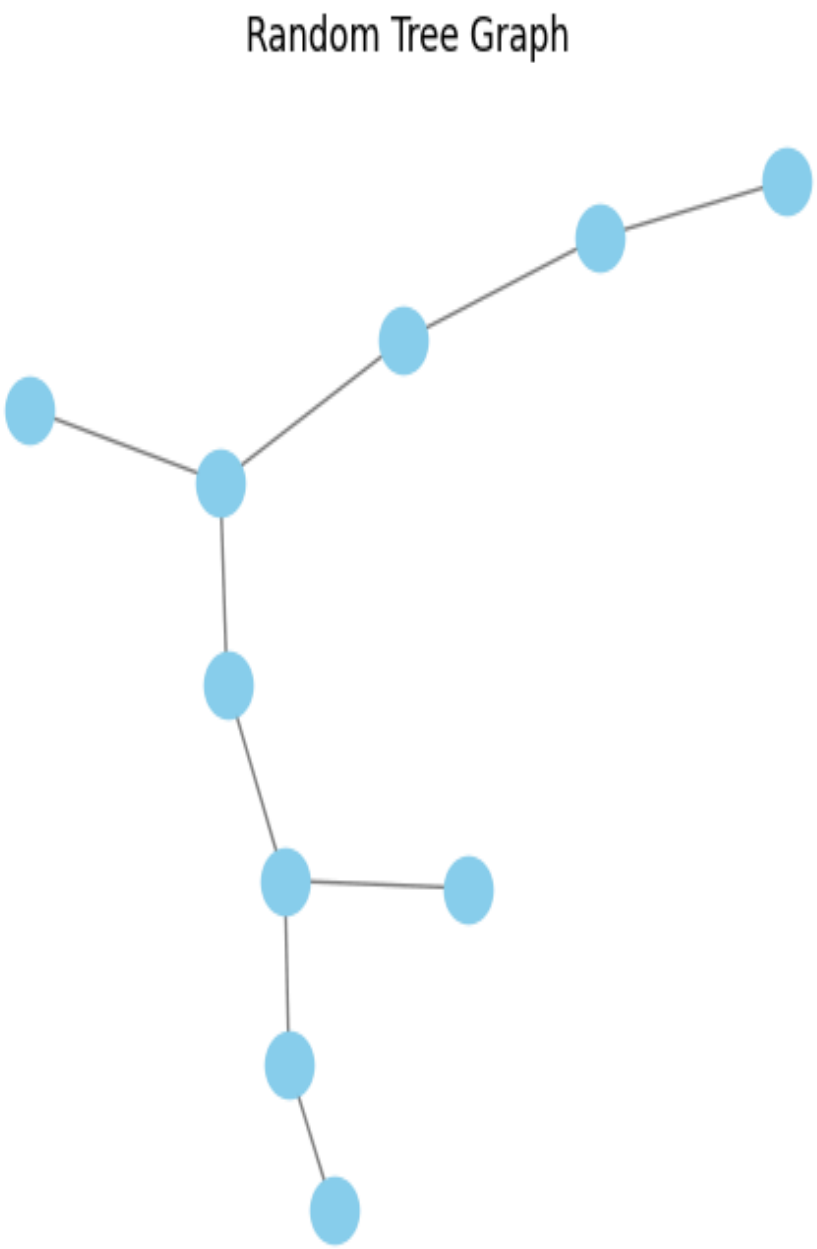

Fig. 13.4 Random Tree Graph

In the code snippet above, we generate a random tree

visualize the graph using the library. `networkx.random_tree` `matplotlib`

13.1.4. Stochastic Block Model

The stochastic block model is a generative model for random graphs that captures the community structure of the graph. In this model, nodes are divided into blocks or communities, and edges are generated between nodes based on the block they belong to. The library provides a function to generate random graphs using the stochastic block model. `networkx` `networkx.stochastic_block_model`

```
# Generate a random graph with 2 blocks of 20
G = nx.stochastic_block_model([20, 20], [
                                        [0
                                         |


                                      ], see


# Draw the graph
plt.figure(figsize=(6, 6))
plt.title("Stochastic Block Model")
pos = nx.spring_layout(G, seed=42)
nx.draw_networkx_nodes(G, pos, node_size=300,
nx.draw_networkx_edges(G, pos, width=1.0, alph
plt.axis("off")
plt.show()
```

Saltar al contenido principal

This code gives us the following graph:

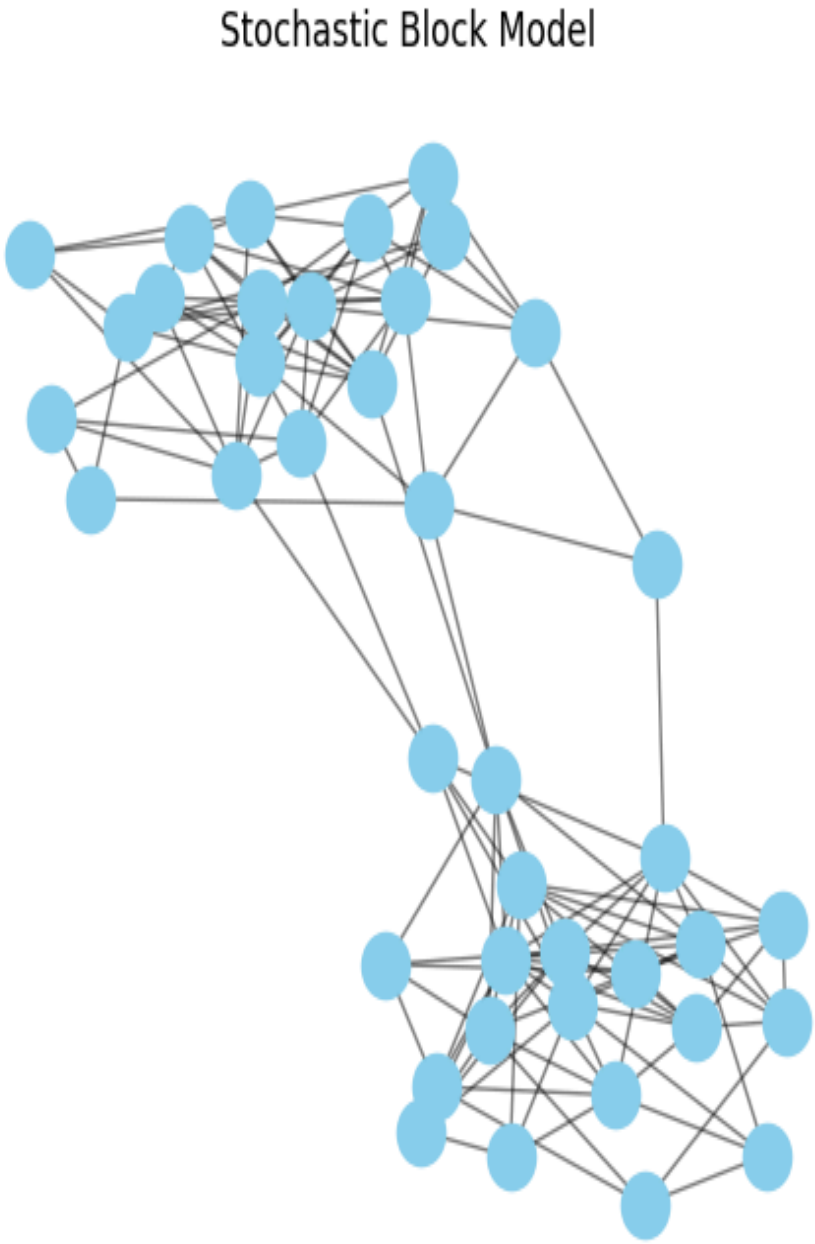

Fig. 13.5 Stochastic Block Model

Saltar al contenido principal

In the code snippet above, we generate a random graph with 2 blocks of 20 nodes each using the stochastic block model. We then visualize the graph using the library. `matplotlib`

13.2. Graph Properties

Once we have generated a graph, we can analyze its properties to gain insights into its structure. Some important graph properties that can be computed using the library include: `networkx`

- **Number of Nodes and Edges**: The number of nodes and edges in a graph can be computed using the and functions, respectively. `networkx.number_of_nodes` `networkx.number_of_edges`
- **Is Connected**: A graph is connected if there is a path between every pair of nodes in the graph. We can check if a graph is connected using the function. `networkx.is_connected`
- **Does Not Have Cycles**: A graph is acyclic if it does not contain any cycles. We can check if a graph is acyclic using the function. `networkx.is_directed_acyclic_graph`
- **Radius and Diameter**: The radius of a graph is

Saltar al contenido principal

graph, while the diameter is the maximum eccentricity of any node in the graph. We can compute the radius and diameter using the and functions, respectively. Note that eccentricity is the maximum distance between a node and all other nodes in the graph. Also, this function only works for connected graphs. `networkx.radius` `networkx.diameter`

- **Average Degree**: The average degree of a graph is the average number of edges incident to each node in the graph. We can compute the average degree using the function. `networkx.average_degree_connectivity`
- **Degree Distribution**: The degree distribution of a graph is the probability distribution of the degrees of the nodes in the graph. We can compute the degree distribution using the function. `networkx.degree_histogram`
- **Average Shortest Path Length**: The average shortest path length of a graph is the average distance between pairs of nodes in the graph. We can compute the average shortest path length using the function. `networkx.average_shortest_path_length`

Saltar al contenido principal

13.3. Conclusion

In this section, we have discussed how to generate synthetic graphs using the library. We have explored different models for generating random graphs, such as the Erdős-Rényi model, the Barabási-Albert model, and the Watts-Strogatz model. We have also seen how to generate tree graphs and stochastic block model graphs. Finally, we have discussed some important graph properties that can be computed using the library. These properties can help us analyze the structure of a graph and gain insights into its properties.`networkx``networkx`

13.4. Exercises

13.4.1. Exercise 1

After reading the previous section, you should be able to create your first Jupyter-Notebook and write a report about all the things you have learned in this section. In order to have a good structure, we recommend you to follow exactly the same structure as in this notebook.

Saltar al contenido principal

13.4.2. Exercise 2

> **Note**
>
> NO PUEDE ESTAR DESCONECTADO EL GRAFO. USAR LA MISMA POS PARA EL DRAW, SI NO, SE CASTIGARÁ. JUSTIFICACIÓN EXTENSA Y CON SENTIDO. NADA DE COSAS OBVIAS.

Pay attention to Stochastic Block Model, and try to generate two graph with 4 blocks of 10 nodes each. The first one, must have the lowest probability of connection between blocks, and the second one, the highest probability of connection between blocks. Then try to visualize the graphs and compare them. In the comparation, try to think about the differences in the structure of the graphs in terms of what would happen if the communities were real people and they try to communicate with each other. It would be easier to communicate in the first or in the second graph? Why?

> **Note**
>
> You need to use you own seed in order to not have the same graph as your classmates.

Saltar al contenido principal

13.4.3. Exercise 3

> **Note**
>
> SOLO LOS DOS SBMS GENERADOS EN EL EJERCICIO 2.

For each graph you have generated in the previous exercise, try to compute the following properties:

- Number of Nodes and Edges
- Is Connected
- Does Not Have Cycles
- Radius and Diameter
- Average Degree
- Degree Distribution (You need to plot the histogram)
- Average Shortest Path Length

Try to have a cell for each graph. One template could be the following:

```
# Number of Nodes and Edges
print(f"Number of Nodes: {nx.number_of_nodes(G
print("\n")
print(f"Number of Edges: {nx.number_of_edges(G
```

Saltar al contenido principal

Previous
12. NetworkX

Next
14. Graph Exploration BFS